NAME: Afifa Syed
T23 2511139
SUBJECT: ASD&D
DATE: 29/7/26

Experiment 2:

Aim: To containerise the application using docker

This experiment maps to LO4.

LO4: Containerize, orchestrate, and deploy applications using Docker, Docker Compose, Kubernetes, and cloud infrastructure tools.

Theory:

Docker is an open-source containerization platform that enables applications to run in isolated environments called **containers**. A container includes the application along with all its required libraries, dependencies, and configuration files, ensuring consistent execution across different systems. Since containers share the host operating system's kernel, they are lightweight, portable, and start much faster than traditional virtual machines.

A Docker image is a read-only template that contains everything needed to create a container, including the operating system, software packages, and application files. Images are stored in repositories such as Docker Hub and can be downloaded using the docker pull command. Once an image is available locally, it can be used to create one or more containers.

Docker provides several commands for managing images and containers. The docker --version command is used to verify the installed Docker version, while docker images display all locally available Docker images. The docker search command allows users to search Docker Hub for official or community-maintained images, and docker pull downloads the required image from the repository.

Containers are created and started using the docker run command. Running a container interactively (docker run -it ubuntu) opens a Linux shell inside the container, allowing users to execute Linux commands as if they were working on a standalone Ubuntu system. Commands such as pwd display the current working directory, ls lists the files and directories, mkdir creates a new directory, cd changes the current directory, touch creates an empty file, echo writes text into a file, and cat displays the contents of the file.

These basic Docker and Linux commands help users understand how containers work, how to interact with a running container, and how to perform file and directory operations within an isolated environment. This forms the foundation for deploying, testing, and managing applications using Docker in real-world DevOps workflows.

NAME: Afifa Syed
T23 2511139
SUBJECT: ASD&D
DATE: 29/7/26

Docker Installation Steps

1. Check system requirements: Ensure your operating system supports Docker and virtualization.
2. Enable virtualization (Windows): Turn on Intel VT-x or AMD-V in the BIOS/UEFI settings.
3. Install WSL 2 (Windows only): WSL 2 provides the Linux environment required by Docker Desktop.
4. Download and install Docker Desktop: Install Docker Desktop from the official Docker website and follow the setup instructions.
5. Restart your computer: Restart if prompted to complete the installation.
6. Launch Docker Desktop: Open Docker Desktop and wait for the Docker Engine to start.

What is a docker image?

A Docker image is a read-only template used to create Docker containers. It contains everything required to run an application, including the operating system, application code, libraries, dependencies, configuration files, and runtime environment. Docker images are stored in repositories such as Docker Hub and can be downloaded using the docker pull command. Since images are portable and platform-independent, they ensure that applications run consistently across different environments.

A Docker image itself cannot execute applications. When the docker run command is used, Docker creates a container, which is a running instance of the image. Multiple containers can be created from the same image, making Docker an efficient solution for application deployment, testing, and scaling. Images are also built in layers, which reduces storage requirements and speeds up image downloads and updates by reusing common layers.

What is containerization?

Containerization is a virtualization technology that packages an application along with all its dependencies, libraries, configuration files, and runtime environment into a single, isolated unit called a container. This ensures that the application runs consistently across different environments, regardless of the underlying operating system or hardware.

Unlike traditional virtual machines, containers share the host operating system's kernel, making them lightweight, fast, and resource-efficient. Containers start within seconds, require less

NAME: Afifa Syed
T23 2511139
SUBJECT: ASD&D
DATE: 29/7/26

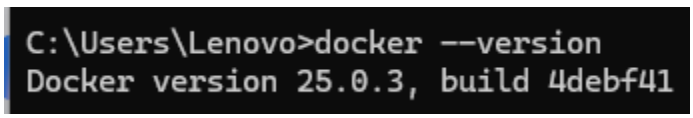
memory, and allow multiple applications to run independently on the same system without conflicts. Docker is one of the most widely used platforms for containerization, enabling developers to build, deploy, and manage applications efficiently. Containerization improves application portability, scalability, and reliability, making it an essential technology in modern software development and DevOps practices.

What is the difference between docker image and docker container?

Docker Image	Docker Container
A read-only template used to create containers.	A running instance of a Docker image.
Contains application code, libraries, dependencies, and configuration files.	Contains the running application along with its processes and data.
Cannot execute applications by itself.	Executes the application inside an isolated environment.
Created using a Dockerfile.	Created using the docker run command.
Stored in image repositories such as Docker Hub.	Exists only while it is created on the host system.
Multiple containers can be created from a single image.	Each container is an independent instance of the image.
Read-only and cannot be modified directly.	Read-write; changes can be made while the container is running.
Example: ubuntu, nginx, python, mysql images.	Example: A running Ubuntu container created using docker run -it ubuntu.

Implementation:

1)check docker version



2)images

NAME: Afifa Syed
T23 2511139
SUBJECT: ASD&D
DATE: 29/7/26

```
C:\Users\Lenovo>docker images
REPOSITORY   TAG       IMAGE ID       CREATED       SIZE
ubuntu       latest    de7345b16e94   2 weeks ago   100MB
ubuntu       <none>    ca2b0f26964c   2 years ago   77.9MB
```

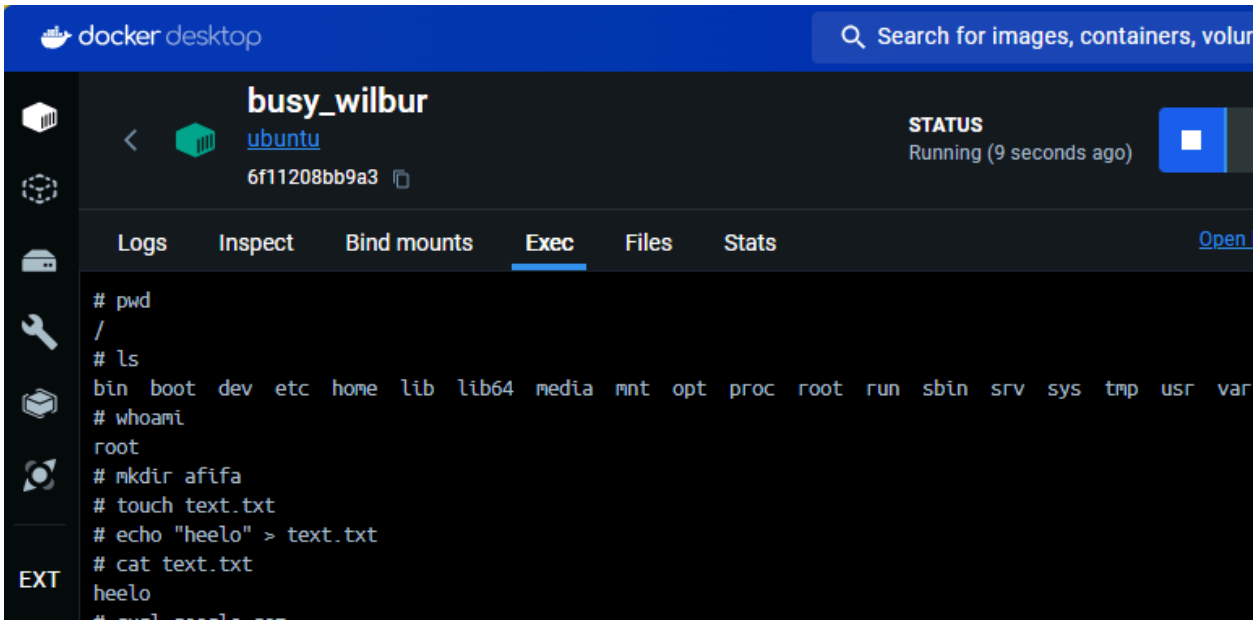
3)

```
C:\Users\Lenovo>docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
Digest: sha256:3131b4cc82a783df6c9df078f86e01819a13594b865c2cad47bd1bca2b7063bb
Status: Image is up to date for ubuntu:latest
docker.io/library/ubuntu:latest

What's Next?
  1. Sign in to your Docker account → docker login
  2. View a summary of image vulnerabilities and recommendations → docker scout quickview ubuntu
```

4)

NAME: Afifa Syed
T23 2511139
SUBJECT: ASD&D
DATE: 29/7/26




```
C:\Users\Lenovo>docker run -it ubuntu
root@6f11208bb9a3:/# pwd
/
```

```
root@b5ab4f3c3afc:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@b5ab4f3c3afc:/# cd home
```

```
root@6f11208bb9a3:/# cd
root@6f11208bb9a3:~# cd home
bash: cd: home: No such file or directory
root@6f11208bb9a3:~# mkdir 2511139afifa
root@6f11208bb9a3:~# cd 2511139afifa
```

```
root@6f11208bb9a3:~/2511139afifa# touch
touch: missing file operand
Try 'touch --help' for more information.
root@6f11208bb9a3:~/2511139afifa# touch roll_no_139
root@6f11208bb9a3:~/2511139afifa# echo "This is my text" > roll_no_139.txt
root@6f11208bb9a3:~/2511139afifa# read roll_no_139
```

```
root@6f11208bb9a3:~/2511139afifa# exit
exit
```

NAME: Afifa Syed
T23 2511139
SUBJECT: ASD&D
DATE: 29/7/26

```
C:\Users\Lenovo>docker run -it --name afifa ubun
tu
root@fb0a5b17b549:/# pwd
/
root@fb0a5b17b549:/# ls
bin   etc   lib64  opt   run   sys  var
boot  home  media  proc  sbin  tmp
dev   lib   mnt    root  srv   usr
root@fb0a5b17b549:/# whoami
root
root@fb0a5b17b549:/# uname -a
Linux fb0a5b17b549 5.15.146.1-microsoft-standard
-WSL2 #1 SMP Thu Jan 11 04:09:03 UTC 2024 x86_64
 GNU/Linux
root@fb0a5b17b549:/#
```

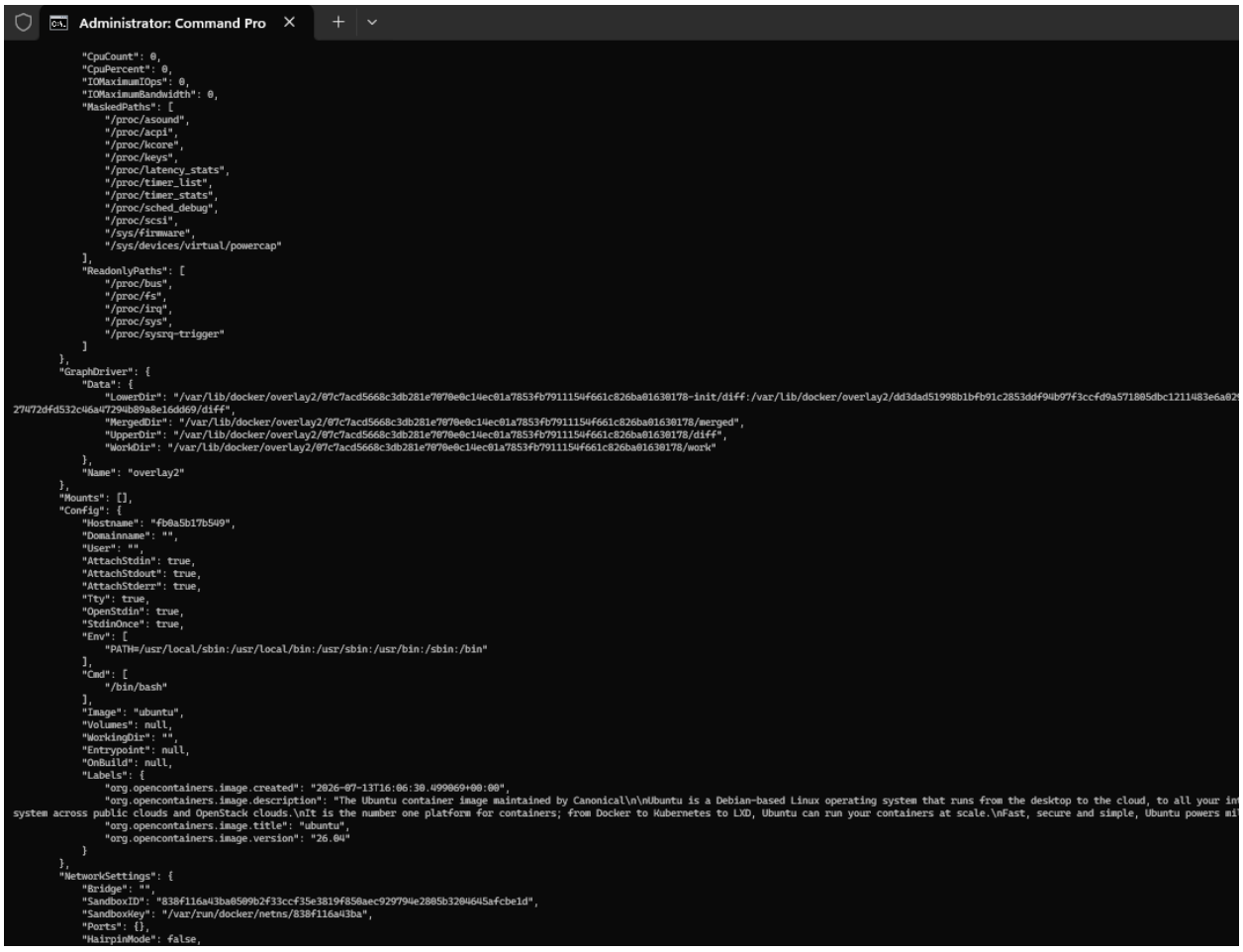
```
C:\Users\Lenovo>docker stop afifa
afifa
```

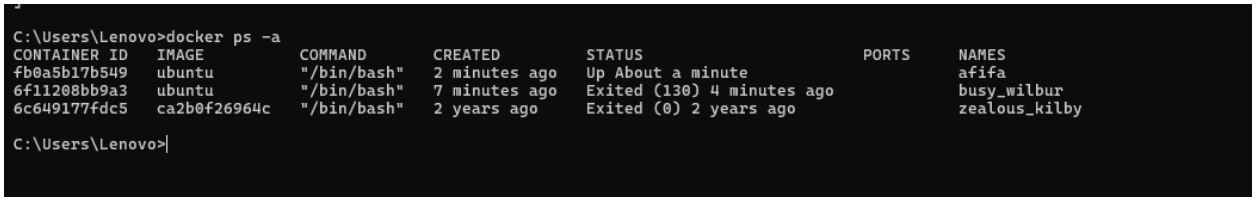
```
C:\Users\Lenovo>docker start afifa
afifa
```

NAME: Afifa Syed
T23 2511139
SUBJECT: ASD&D
DATE: 29/7/26

```
C:\Users\Lenovo>docker inspect afifa
[
    {
        "Id": "fb0a5b17b549342672bef2da31fd9365e832abdc927f2cd763364e8fff218800",
        "Created": "2026-07-29T02:38:30.8919594Z",
        "Path": "/bin/bash",
        "Args": [],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 986,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2026-07-29T02:39:56.5675329Z",
            "FinishedAt": "2026-07-29T02:39:34.0596680Z"
        },
        "Image": "sha256:de7345b16e942e22044c6ba053020ec85ae879984860a9918517d54eb6cef851",
        "ResolvConfPath": "/var/lib/docker/containers/fb0a5b17b549342672bef2da31fd9365e832abdc927f2cd763364e8fff218800/resolv.conf",
        "HostnamePath": "/var/lib/docker/containers/fb0a5b17b549342672bef2da31fd9365e832abdc927f2cd763364e8fff218800/hostname",
        "HostsPath": "/var/lib/docker/containers/fb0a5b17b549342672bef2da31fd9365e832abdc927f2cd763364e8fff218800/hosts",
        "LogPath": "/var/lib/docker/containers/fb0a5b17b549342672bef2da31fd9365e832abdc927f2cd763364e8fff218800/fb0a5b17b549342672bef2da31fd9365e832abdc927f2cd763364e8fff218800-json.l
        "Name": "/afifa",
        "RestartCount": 0,
        "Driver": "overlay2",
        "Platform": "linux",
        "MountLabel": "",
        "ProcessLabel": "",
        "AppArmorProfile": "",
        "ExecIDs": null,
        "HostConfig": {
            "Binds": null,
            "ContainerIDFile": "",
            "LogConfig": {
                "Type": "json-file",
                "Config": {}
            },
            "NetworkMode": "default",
            "PortBindings": {},
            "RestartPolicy": {
                "Name": "no",
                "MaximumRetryCount": 0
            },
            "AutoRemove": false,
            "VolumeDriver": "",
            "VolumesFrom": null,
            "ConsoleSize": [
                30,
                48
            ],
```

NAME: Afifa Syed
T23 2511139
SUBJECT: ASD&D
DATE: 29/7/26





NAME: Afifa Syed
T23 2511139
SUBJECT: ASD&D
DATE: 29/7/26

```
C:\Users\Lenovo>docker stats
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.02kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.02kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.02kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.02kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.02kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.02kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.02kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.02kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.02kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.02kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.02kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.02kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.02kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.02kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.02kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.02kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.09kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.09kB / 0B   0B / 0B     1
CONTAINER ID   NAME      CPU %     MEM USAGE / LIMIT   MEM %     NET I/O       BLOCK I/O   PIDS
fb0a5b17b549   afifa     0.00%     928KiB / 7.62GiB    0.01%     1.09kB / 0B   0B / 0B     1
```

```
C:\Users\Lenovo>docker rm -f afifa
afifa

C:\Users\Lenovo>
```

```
C:\Users\15L>docker image prune
WARNING! This will remove all dangling images.
Are you sure you want to continue? [y/N] y
Total reclaimed space: 0B
```

```
C:\Users\15L>docker container ls -a
CONTAINER ID   IMAGE                COMMAND                  CREATED             STATUS                      PORTS                    NAMES
16413dce4ead   ubuntu               "/bin/bash"              10 minutes ago      Exited (0) 7 minutes ago                             mystifying_thompson
dc1393855c11   kuanweilien/anime    "python3 aniGamerPlu…"   11 minutes ago      Exited (255) 11 minutes ago                          recursing_jackson
9a62fe36676e   kuanweilien/anime    "python3 aniGamerPlu…"   12 minutes ago      Exited (255) 12 minutes ago                          lucid_joliot
d409032c79f8   kuanweilien/anime    "python3 aniGamerPlu…"   14 minutes ago      Exited (255) 12 minutes ago                          stupefied_easley
b5ab4f3c3afc   ubuntu               "/bin/bash"              About an hour ago   Exited (0) 25 minutes ago                            jolly_lehmann
bbaf42491e8e   my_custom_app        "python app.py"          27 hours ago        Exited (255) 2 hours ago    5000/tcp                 my_web_app
fe9d430fb8b3   my-flask-app         "python app.py"          28 hours ago        Exited (255) 2 hours ago    0.0.0.0:5000->5000/tcp   distracted_carson
b125f8a6a583   ubuntu               "/bin/bash"              47 hours ago        Exited (0) 47 hours ago                              ubuntu-test
5fde9bfdd162   ca2b0f26964c         "bash -c 'echo Hello…"   4 months ago        Exited (0) 4 months ago                              frosty_kepler
6587c9c4a2bf   nginx                "/docker-entrypoint.…"   4 months ago        Exited (0) 4 months ago                              charming_franklin
adefa716cb32   hello-world          "/hello"                 4 months ago        Exited (0) 4 months ago                              jolly_swanson
77ea50afe062   ca2b0f26964c         "bash -c 'echo Hello…"   4 months ago        Exited (0) 4 months ago                              agitated_agnesi
56c98c2698d7   ca2b0f26964c         "bash"                   4 months ago        Exited (0) 4 months ago                              pedantic_golick
```

NAME: Afifa Syed
T23 2511139
SUBJECT: ASD&D
DATE: 29/7/26

```
C:\Users\Lenovo>docker info
Client:
 Version:    25.0.3
 Context:    default
 Debug Mode: false
 Plugins:
  buildx: Docker Buildx (Docker Inc.)
    Version:  v0.12.1-desktop.4
    Path:     C:\Program Files\Docker\cli-plugins\docker-buildx.exe
  compose: Docker Compose (Docker Inc.)
    Version:  v2.24.6-desktop.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-compose.exe
  debug: Get a shell into any image or container. (Docker Inc.)
    Version:  0.0.24
    Path:     C:\Program Files\Docker\cli-plugins\docker-debug.exe
  dev: Docker Dev Environments (Docker Inc.)
    Version:  v0.1.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-dev.exe
  extension: Manages Docker extensions (Docker Inc.)
    Version:  v0.2.22
    Path:     C:\Program Files\Docker\cli-plugins\docker-extension.exe
  feedback: Provide feedback, right in your terminal! (Docker Inc.)
    Version:  v1.0.4
    Path:     C:\Program Files\Docker\cli-plugins\docker-feedback.exe
  init: Creates Docker-related starter files for your project (Docker Inc.)
    Version:  v1.0.1
    Path:     C:\Program Files\Docker\cli-plugins\docker-init.exe
  sbom: View the packaged-based Software Bill Of Materials (SBOM) for an image (Anchore Inc.)
    Version:  0.6.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-sbom.exe
  scout: Docker Scout (Docker Inc.)
    Version:  v1.5.0
    Path:     C:\Program Files\Docker\cli-plugins\docker-scout.exe

Server:
 Containers: 2
  Running: 0
  Paused: 0
  Stopped: 2
 Images: 2
 Server Version: 25.0.3
 Storage Driver: overlay2
  Backing Filesystem: extfs
  Supports d_type: true
  Using metacopy: false
  Native Overlay Diff: true
  userxattr: false
 Logging Driver: json-file
 Cgroup Driver: cgroupfs
 Cgroup Version: 1
```

```
C:\Users\Lenovo>docker history ubuntu
IMAGE          CREATED       CREATED BY                                      SIZE      COMMENT
de7345b16e94   2 weeks ago   umoci raw add-layer --image /home/buildd/roc…   179B      Add rock control metadata
<missing>      2 weeks ago   umoci config --image /home/buildd/rockcraft-…   0B        Set annotations
<missing>      2 weeks ago   umoci config --image /home/buildd/rockcraft-…   0B        Set labels
<missing>      2 weeks ago   umoci config --image /home/buildd/rockcraft-…   0B        Set default PATH for bare-base
rock
<missing>      2 weeks ago   umoci config --image /home/buildd/rockcraft-…   0B        Set default commands
<missing>      2 weeks ago   umoci config --image /home/buildd/rockcraft-…   0B        Set entrypoint
<missing>      2 weeks ago   umoci raw add-layer --image /home/buildd/roc…   100MB
```

NAME: Afifa Syed
T23 2511139
SUBJECT: ASD&D
DATE: 29/7/26

```
C:\Users\Lenovo>docker network inspect bridge
[
    {
        "Name": "bridge",
        "Id": "014a44c91d21d91925f64ac524d3287678ad4399f74bb2e148ba0b9db28ab24e",
        "Created": "2026-07-29T02:24:00.7311205Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.17.0.0/16",
                    "Gateway": "172.17.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Containers": {},
        "Options": {
            "com.docker.network.bridge.default_bridge": "true",
            "com.docker.network.bridge.enable_icc": "true",
            "com.docker.network.bridge.enable_ip_masquerade": "true",
            "com.docker.network.bridge.host_binding_ipv4": "0.0.0.0",
            "com.docker.network.bridge.name": "docker0",
            "com.docker.network.driver.mtu": "1500"
        },
        "Labels": {}
    }
]
```

```
C:\Users\Lenovo>docker network create afifa
b7612e0a0d678051a290a35bc6d57239bffb7ba92fc28bab974d00815554c1a7

C:\Users\Lenovo>
```

Conclusion:

In this experiment, we learned the fundamentals of Docker by performing basic image and container operations. We verified the Docker installation, searched for and downloaded Docker images from Docker Hub, listed locally available images, and created an Ubuntu container using

a Docker image. Inside the container, we executed basic Linux commands to navigate directories, create folders and files, write data to a file, and verify its contents. This experiment helped us understand the concepts of Docker images, containers, and containerization, as well

NAME: Afifa Syed
T23 2511139
SUBJECT: ASD&D
DATE: 29/7/26

as how Docker provides an isolated and consistent environment for running applications efficiently.